

Relatório

Trabalho 2 – Remote Method Invocation (RMI)

- Raissa Ivyna - 553679
 - Francisca Ariane - 554930
-

1. Introdução

Este relatório descreve a implementação do Trabalho 2 da disciplina de Sistemas Distribuídos, cujo objetivo é reimplementar o sistema de gerenciamento de produtos veterinários e estoques — desenvolvido no Trabalho 1 — utilizando comunicação cliente-servidor via Remote Method Invocation (RMI).

A implementação seguiu o protocolo de requisição-resposta, com separação clara entre cliente e servidor: o servidor foi implementado em C++ e o cliente em Java, demonstrando interoperabilidade entre linguagens por meio de um protocolo baseado em JSON.

Os três métodos centrais do protocolo foram implementados em ambas as linguagens: doOperation (cliente), getRequest e sendReply (servidor), conforme exigido pelo enunciado.

2. Divisão do Trabalho

Servidor – C++ (Ariane)	Cliente – Java (Raissa)
SocketUtils: getRequest e sendReply	RequestReplyProtocol: doOperation
RMIServidor: loop de atendimento	ClienteRMI: chamadas remotas
Expedidor: roteamento por RemoteObjectRef	RemoteObjectRef: referência ao objeto remoto
ProdutoSkeleton e EstoqueSkeleton	Serialização manual de JSON
ProdutoServico e EstoqueServico	Testes de todos os métodos remotos

3. Arquitetura do Sistema

O sistema segue o padrão Skeleton-Stub. O fluxo de uma chamada remota é:

- O cliente monta a requisição JSON com RemoteObjectRef, metodold e parametros.
- doOperation (Java) abre um socket TCP, envia o JSON como linha de texto e bloqueia aguardando a resposta.
- O servidor recebe via getRequest (C++), faz parse do JSON e repassa ao Expedidor.
- O Expedidor lê o NomeObjeto do RemoteObjectRef e encaminha ao Skeleton correto.

- O Skeleton faz unmarshalling dos parâmetros, invoca o serviço e serializa o resultado em JSON.
- `sendReply` (C++) envia a resposta. `doOperation` (Java) recebe, retorna os bytes ao `ClienteRMI`.

4. Implementação do Servidor (C++)

4.1 SocketUtils – Camada de Comunicação

A classe `SocketUtils` encapsula todos os sockets TCP/IP. Nenhuma outra classe do servidor manipula sockets diretamente. Implementa quatro métodos:

- `criarSocketServidor(porta)`: realiza bind e listen, chamado uma única vez na inicialização.
- `getRequest(serverSocket, clientSocketOut)`: aceita conexão e recebe a requisição JSON do cliente.
- `sendReply(clientSocket, resposta)`: serializa e envia a resposta JSON, então fecha a conexão.
- `doOperation(host, porta, requisicao)`: usado pelo cliente C++ de teste — cria socket, conecta, envia e recebe.

4.2 RMIServidor

O `RMIServidor` inicializa o servidor com `criarSocketServidor` e mantém um loop infinito chamando `getRequest` para receber requisições e `sendReply` para responder. O Expedidor é instanciado fora do loop para manter o estado do repositório entre requisições.

4.3 RemoteObjectRef e Expedidor

Cada requisição contém um `RemoteObjectRef` serializado em JSON com dois campos: `objetoId` (identificador numérico da instância) e `NomeObjeto` (nome do serviço remoto). O Expedidor lê o `NomeObjeto` e encaminha para o Skeleton correspondente:

```
"ProdutoService" → ProdutoSkeleton
"EstoqueServico" → EstoqueSkeleton
```

4.4 Skeletons

Cada Skeleton recebe o Request, faz o unmarshalling dos parâmetros lendo o JSON (passagem por valor), invoca o serviço e retorna o resultado serializado. O `EstoqueSkeleton` recebe uma referência ao mesmo `ProdutoService` do `ProdutoSkeleton`, garantindo consistência entre os dois serviços.

5. Implementação do Cliente (Java)

5.1 RequestReplyProtocol – doOperation

A classe `RequestReplyProtocol` implementa o método `doOperation`, que é o equivalente Java do `SocketUtils` do servidor:

- Recebe um `RemoteObjectRef` (com host, porta e nome do objeto), o `metodoId` e os argumentos em `byte[]`.

- Abre um socket TCP para o host e porta informados no RemoteObjectRef.
- Envia o JSON da requisição como linha de texto via PrintWriter (out.println).
- Bloqueia em in.readLine() aguardando a resposta do servidor — comportamento síncrono, como uma chamada local.
- Fecha o socket automaticamente via try-with-resources e retorna os bytes da resposta.

O uso de `byte[]` como tipo de retorno segue a assinatura exigida pelo enunciado: `public byte[] doOperation(RemoteObjectRef o, int methodId, byte[] arguments)`, e permite que a camada de comunicação seja independente do formato dos dados.

5.2 RemoteObjectRef (Java)

A classe `RemoteObjectRef` do cliente Java encapsula host, porta e nome do objeto remoto — equivalente ao `RemoteObjectRef` do servidor C++. Ela representa a passagem por referência: o cliente não possui o objeto, apenas uma referência a ele, e o protocolo a usa para saber onde conectar.

5.3 ClienteRMI

O `ClienteRMI` monta as requisições JSON manualmente e chama o método utilitário `chamar()`, que constrói o `RemoteObjectRef`, serializa o JSON no formato exato esperado pelo servidor e invoca `protocolo.doOperation()`. Os 18 testes cobrem os dois objetos remotos (`ProdutoService` e `EstoqueServico`), casos de sucesso, erro e método inexistente.

5.4 Interoperabilidade C++ e Java

O cliente Java envia exatamente o mesmo formato JSON que o cliente C++ de teste, e o servidor C++ responde sem distinção de linguagem. Isso demonstra que o protocolo baseado em JSON funciona como representação externa de dados independente de linguagem, conforme proposto pelo enunciado.

6. Modelo de Domínio

6.1 Hierarquia de Classes ("é-um")

Classe	Herda de	Atributos Adicionais
Produto	(base)	id, nome, preco, fabricante
ProdutoVeterinario	Produto	registroMapa, especieAlvo, viaAdministracao
ProdutoBiologico	ProdutoVeterinario	tipoAgente, sorotipo, numDoses
VacinaPerecivel	ProdutoBiologico	dataValidade, requisitoArmazenamento, temperaturaMin/Max
VacinaNaoPerecivel	ProdutoBiologico	requisitoArmazenamento
ProdutoQuimioterapico	ProdutoVeterinario	substanciaAtiva, concentracao

6.2 Composições "tem-um"

- Estoque tem Produto: a classe Estoque possui um vector<Produto*> produtos, representando os itens armazenados.
- EstoqueService tem Estoque: a classe EstoqueService possui um vector<Estoque> estoques, gerenciando múltiplos estoques.

7. Métodos para Invocação Remota

7.1 ProdutoService

Método	Parâmetros	Descrição
buscarPorId	{ id: int }	Retorna o produto com o id informado.
listarTodos	(nenhum)	Retorna todos os produtos cadastrados.
buscarPorEspecie	{ especie: string }	Filtra produtos veterinários pela espécie alvo.
calcularValorTotal	(nenhum)	Soma os preços de todos os produtos.
remover	{ id: int }	Remove o produto com o id informado.

7.2 EstoqueService

Método	Parâmetros	Descrição
criarEstoque	{ local: string }	Cria um novo estoque com o local informado.
listarEstoques	(nenhum)	Lista todos os estoques com total de produtos.
entradaProduto	{ estoqueId, produtoId }	Adiciona produto a um estoque (buscado do ProdutoService).
saidaProduto	{ estoqueId, produtoId }	Remove produto de um estoque.
alertarVencidos	(nenhum)	Lista produtos vencidos presentes nos estoques.

8. Passagem de Parâmetros

8.1 Passagem por Valor – JSON

Os parâmetros e resultados são transmitidos por valor usando JSON como representação externa de dados. Cada classe implementa toJson() para serialização. Uma cópia dos dados é transmitida pela rede, sem referência ao objeto original. O uso de JSON garante interoperabilidade entre C++ e Java sem necessidade de conversão adicional.

8.2 Passagem por Referência – RemoteObjectRef

O campo referenciaObj de cada requisição é um RemoteObjectRef serializado em JSON:

```
{ "objetoId": 1, "NomeObjeto": "ProdutoService" }
```

O cliente não possui o objeto remoto — apenas sua referência. O Expedidor a usa para localizar e invocar o Skeleton correto.

9. Tecnologias Utilizadas

Componente	Tecnologia	Implementação
Servidor	C++17	Implementação manual do middleware RMI
Cliente	Java 17	Interoperabilidade via protocolo JSON
Serialização	nlohmann/json (C++) / manual (Java)	Representação externa de dados independente de linguagem
Transporte	TCP/IP (sockets POSIX / java.net.Socket)	Confiável: garante entrega completa e ordenada das mensagens

10. Conclusão

O trabalho implementou com sucesso um middleware RMI com servidor em C++ e cliente em Java, seguindo o protocolo de requisição-resposta. A camada de sockets foi encapsulada em SocketUtils (C++) e RequestReplyProtocol (Java), expondo ao restante do sistema apenas os métodos doOperation, getRequest e sendReply.